

ROBUSTNESS AND CGAL

Rodrigo Silveira

Discrete and Algorithmic Geometry
Facultat de Matemàtiques i Estadística
Universitat Politècnica de Catalunya

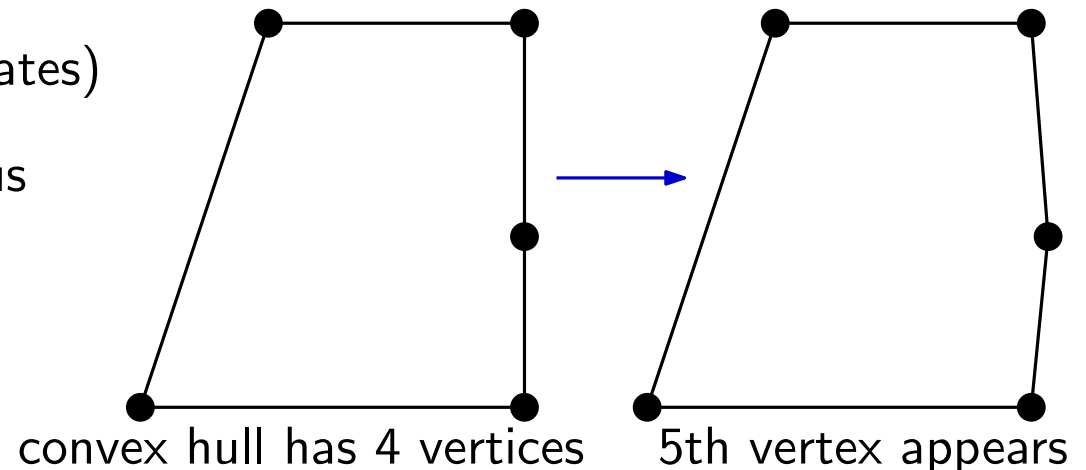
ROBUSTNESS AND CGAL: Introduction

Robustness

(In computer science) The ability of a computer system to cope with errors during execution

Geometric computing and robustness

- A geometric problem is a function
- Inputs are set of points in $\mathbb{R}^2$, $\mathbb{R}^3$ or $\mathbb{R}^d$, or set of geometric objects, e.g., line segments, triangles, polyhedra, etc.
- Output has two parts:
 - discrete part (e.g., a Voronoi diagram, a triangulation, the structure of an arrangement)
 - numerical part (e.g., point coordinates)
- In general, the function is not continuous



Source: *Dependable and Efficient Geometric Computing* by K. Melhorn

ROBUSTNESS AND CGAL: Introduction

Geometric computing and robustness

- Algorithms designed for theoretical machine, the **real RAM**, able to compute with real numbers in constant time.
- Algorithms often neglect degenerate inputs (no three collinear points, no three lines through one point, etc.)
- Real machines use floating point numbers (or bounded integer arithmetic)
- Real inputs often include degeneracies

In practice, things don't work as in theory

Source: *Dependable and Efficient Geometric Computing* by K. Melhorn

ROBUSTNESS AND CGAL: Introduction

Geometric computing and robustness

- Algorithms designed for theoretical machine, the **real RAM**, able to compute with real numbers in constant time.
- Algorithms often neglect degenerate inputs (no three collinear points, no three lines through one point, etc.)
- Real machines use floating point numbers (or bounded integer arithmetic)
- Real inputs often include degeneracies

In practice, things don't work as in theory

Simply using floating-point arithmetic for algorithms designed for the real RAM results in

- Programs that crash (maybe after some assertion failed—if lucky)
- Executions that never end
- Wrong outputs (numerically... and structurally)

Source: *Dependable and Efficient Geometric Computing* by K. Melhorn

ROBUSTNESS AND CGAL: Introduction

Geometric computing and robustness

- Algorithms designed for theoretical machine, the **real RAM**, able to compute with real numbers in constant time.
- Algorithms often neglect degenerate inputs (no three collinear points, no three lines through one point, etc.)
- Real machines use floating point numbers (or bounded integer arithmetic)
- Real inputs often include degeneracies

In practice, things don't work as in theory

Simply using floating-point arithmetic for algorithms designed for the real RAM results in

- Programs that crash (maybe after some assertion failed—if lucky)
- Executions that never end
- Wrong outputs (numerically... and structurally)

Source: *Dependable and Efficient Geometric Computing* by

Time to look at *Classroom Examples of Robustness Problems in Geometric Computations*

ROBUSTNESS AND CGAL

How to deal with this?

Goal: algorithms and implementations that

- work for all inputs
- come with a guarantee: output on an input x is either equal to the mathematically correct output or close to it in a precisely specified way

ROBUSTNESS AND CGAL

Three approaches to reliable geometric computing

- Ad-hoc solutions, for single algorithms
- Arbitrary-precision arithmetic (e.g., GMP, CORE, LEDA)
 - Digits of precision limited only by the memory
 - Implemented for certain number types (integers, rationals, algebraic numbers)
 - No solution for transcendental numbers
 - Slow!
- More efficient: exact geometric computation paradigm (EGC)
 - Implement restricted real-RAM (to the extent needed in computational geometry)
 - Challenge: efficiency (memory and time)
- Approximation (*controlled perturbation*)
 - Compute correct result for slightly perturbed input
 - Challenge: show that approach is widely applicable

ROBUSTNESS AND CGAL

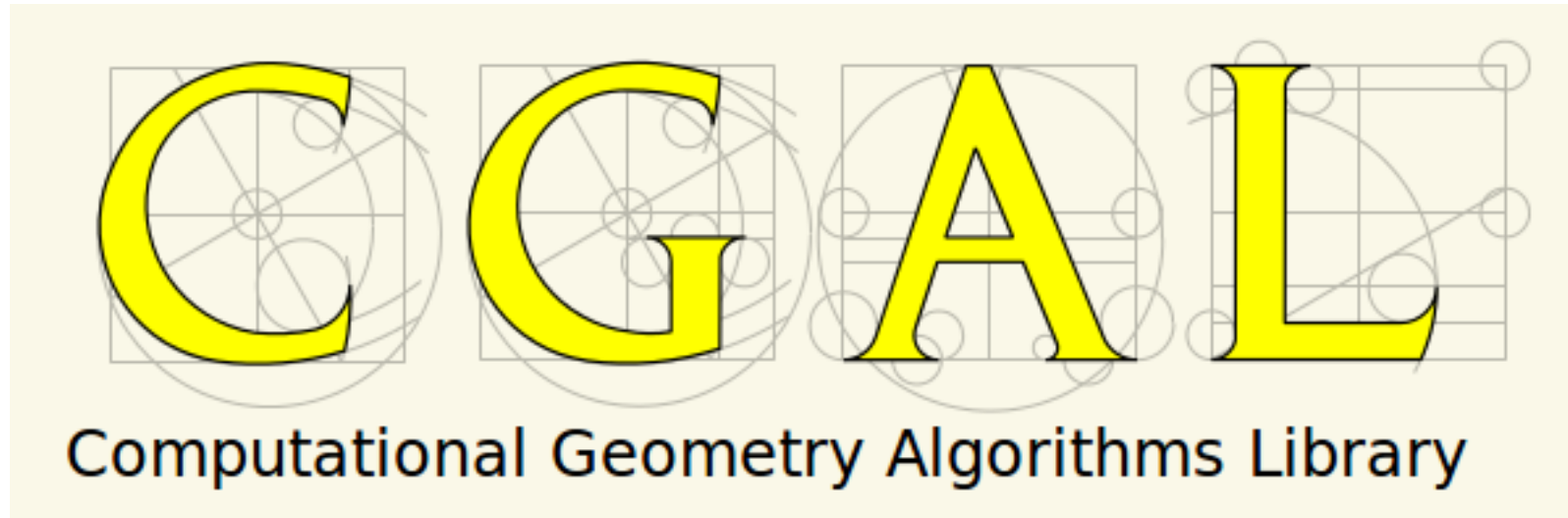
Exact Geometric Computation Paradigm (EGC)

Goal: ensure **correct control flow** of algorithm

- **Exact** evaluation of geometric predicates
 - Functions computing *combinatorial results* from numerical input
 - Example: orientation test, in circle test, intersection test
- Results in exact geometric constructions
 - Intersections, projections, ...
 - If there are any !

Implemented in CGAL

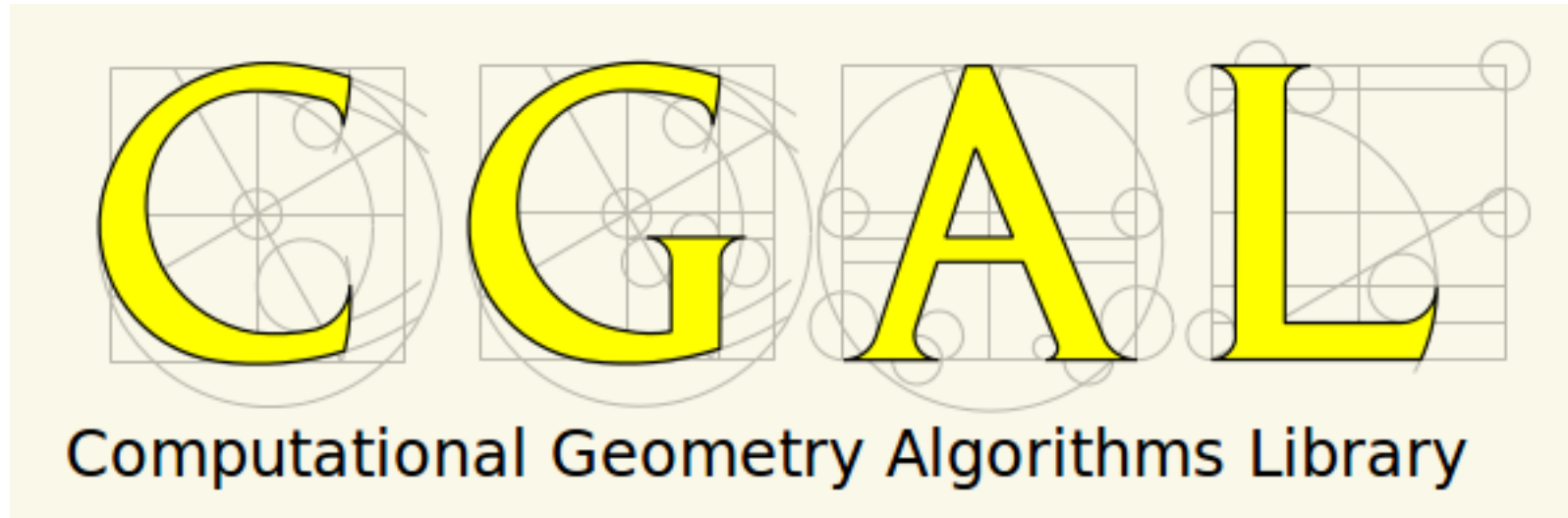
CGAL



Goal: *Make the large body of geometric algorithms developed in the field of computational geometry available for industrial applications*

CGAL project proposal (1996)

CGAL



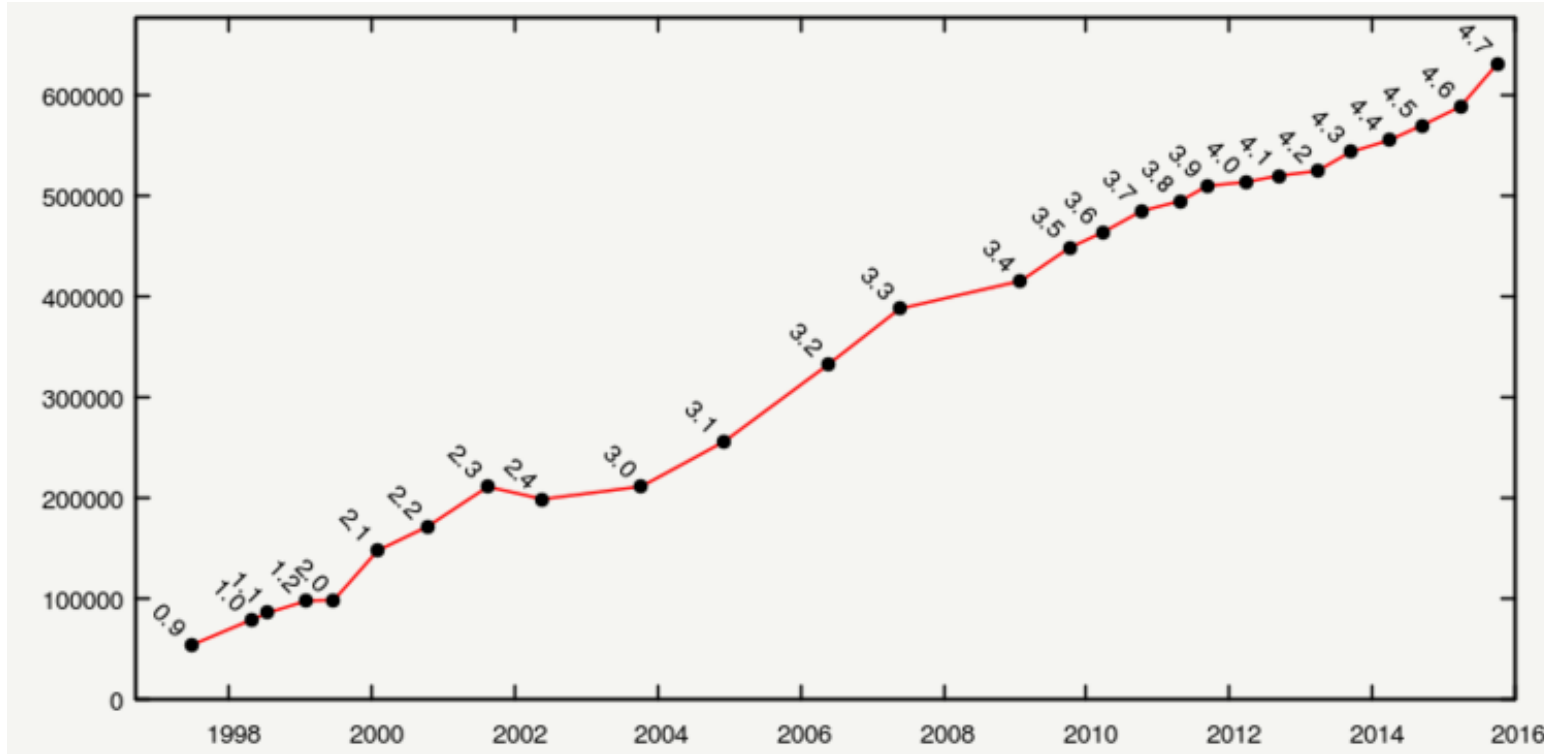
Goal: *Make the large body of geometric algorithms developed in the field of computational geometry available for industrial applications*

CGAL project proposal (1996)

- Project started in 1995
- C++ library
- Developed in many universities
 - About 10 of them still actively contributing

CGAL

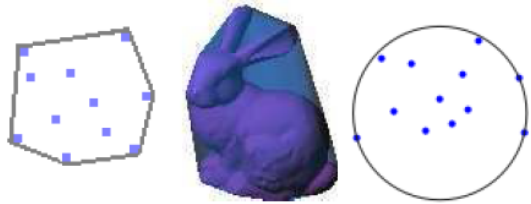
CGAL is big



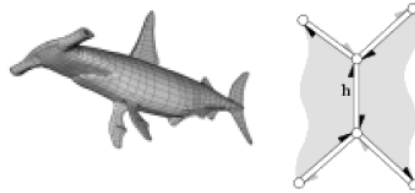
- +600,000 lines of code in latest version (4.7 - October 2015)
- +3,500 manual pages (in 2012)
- Released under 2 licenses: open source (LGPL and GPL) and commercial

CGAL

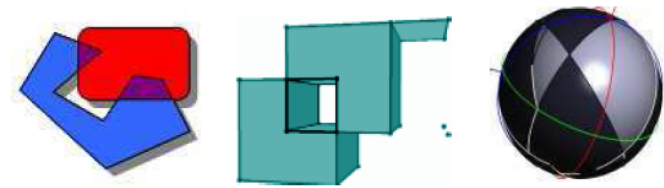
Over 80 packages (plus some support and visualization ones)



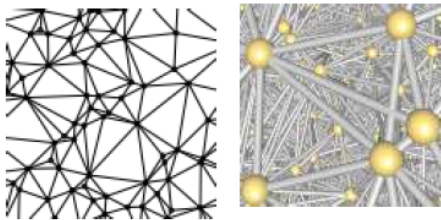
Bounding Volumes



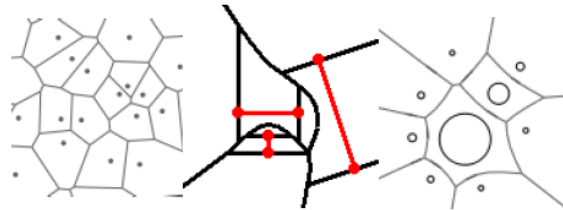
Polyhedral Surface



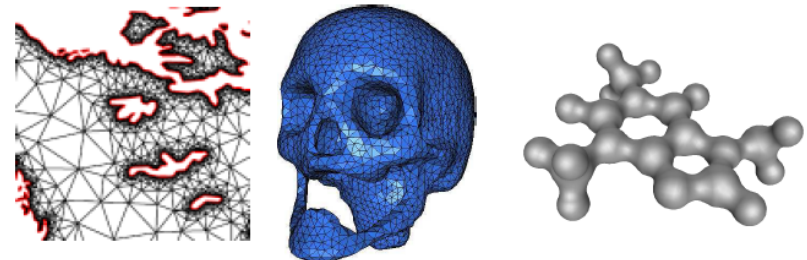
BooleanOperations



Triangulations



Voronoi Diagrams



Mesh Generation



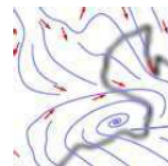
Subdivision



Simplification



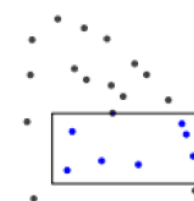
Parametrisation



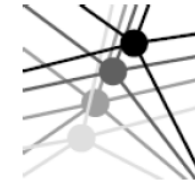
Streamlines



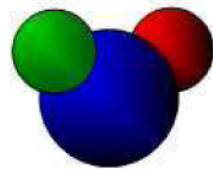
Ridge
Detection



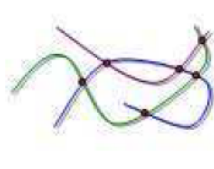
Neighbor
Search



Kinetic
Datastructures



Lower Envelope



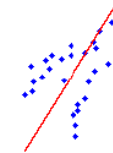
Arrangement



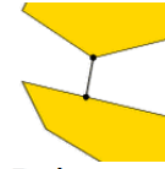
Intersection
Detection



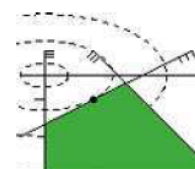
Minkowski
Sum



PCA



Polytope
distance



QP Solver

Some important features

- Reliability
 - Explicitly handles degeneracies
 - Follows the Exact Geometric Computation (EGC) paradigm
- Flexibility
 - Open source library
 - Depends on other libraries (e.g., Boost, GMP, MPFR, Qt, and CORE)
 - Modular structure, e.g., geometry and topology are separated
 - Adaptable and extensible, e.g., data structures can be extended
- Ease of Use
 - Has didactic and exhaustive manuals
 - Follows standard concepts (e.g., C++ and STL)
 - Smooth learning-curve (although steep for many)
 - Active support mailing list

This is how CGAL code looks

File `Convex_hull_2/array_convex_hull_2.cpp`

```
#include <iostream>
#include <CGAL/Exact_predicates_inexact_constructions_kernel.h>
#include <CGAL/convex_hull_2.h>

typedef CGAL::Exact_predicates_inexact_constructions_kernel K;
typedef K::Point_2 Point_2;

int main()
{
    Point_2 points[5] = { Point_2(0,0), Point_2(10,0), Point_2(10,10), Point_2(6,5), Point_2(4,1) };
    Point_2 result[5];

    Point_2 *ptr = CGAL::convex_hull_2( points, points+5, result );
    std::cout << ptr - result << " points on the convex hull:" << std::endl;
    for(int i = 0; i < ptr - result; i++){
        std::cout << result[i] << std::endl;
    }
    return 0;
}
```

This is how CGAL code looks

File Convex_hull_2/array_convex_hull_2.cpp

```
#include <iostream>
#include <CGAL/Exact_predicates_inexact_constructions_kernel.h>
#include <CGAL/convex_hull_2.h>

typedef CGAL::Exact_predicates_inexact_constructions_kernel K;
typedef K::Point_2 Point_2;

int main()
{
    Point_2 points[5] = { Point_2(0,0), Point_2(10,0), Point_2(10,10), Point_2(6,5), Point_2(4,1) };
    Point_2 result[5];

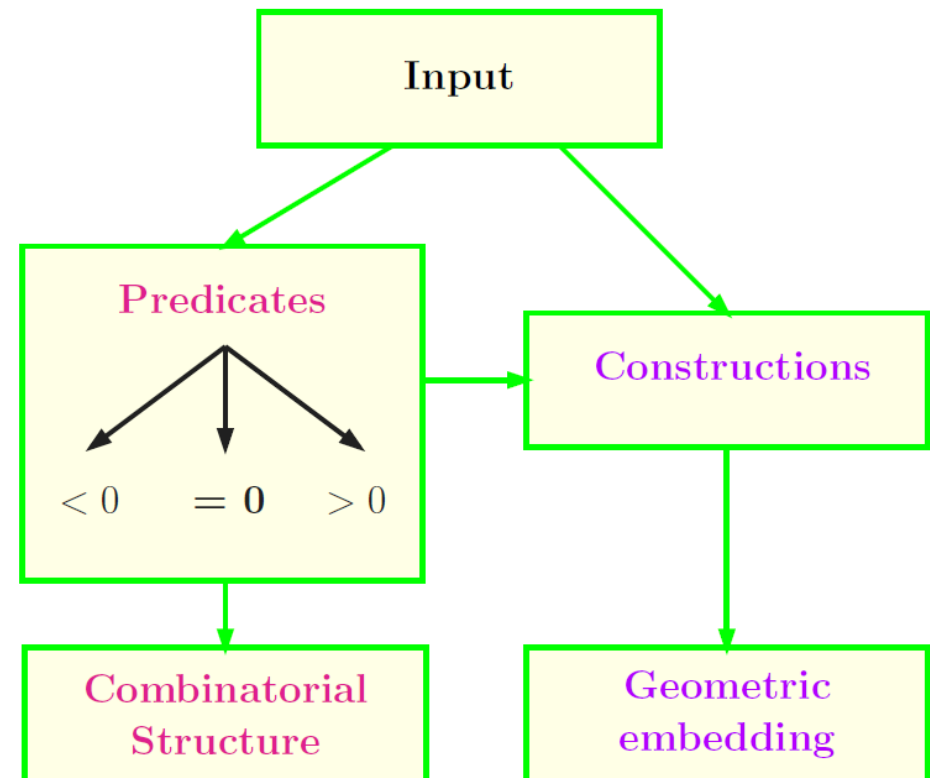
    Point_2 *ptr = CGAL::convex_hull_2( points, points+5, result );
    std::cout << ptr - result << " points on the convex hull:" << std::endl;
    for(int i = 0; i < ptr - result; i++){
        std::cout << result[i] << std::endl;
    }
    return 0;
}
```

Kernel: encapsulates elementary geometric objects and basic computations on them (like predicates)

ROBUSTNESS IN CGAL

Two aspects

- Degenerate cases
 - Explicit treatment of degenerate cases
 - Sometimes using general techniques, like symbolic perturbation
- Numerical robustness
 - Kernels and arithmetics
 - Separation between predicates and constructions

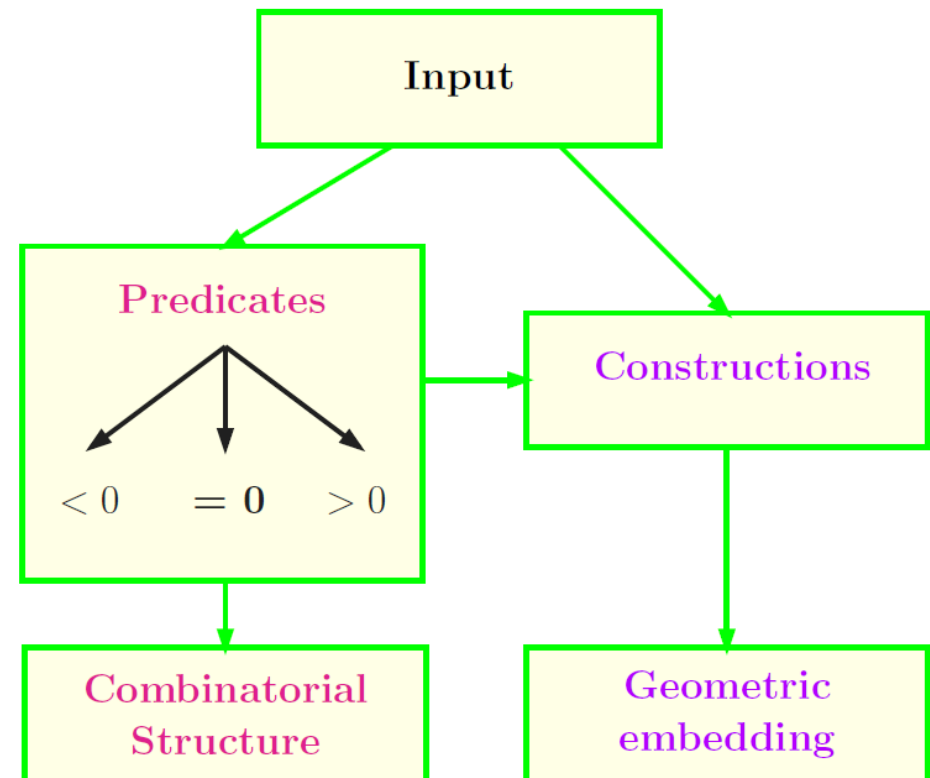
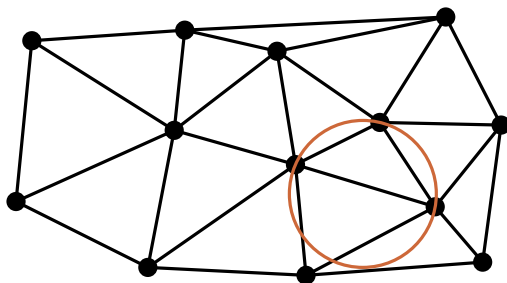


ROBUSTNESS IN CGAL

Two aspects

- Degenerate cases
 - Explicit treatment of degenerate cases
 - Sometimes using general techniques, like symbolic perturbation
- Numerical robustness
 - Kernels and arithmetics
 - Separation between predicates and constructions

Example: to compute the Delaunay triangulations, only two predicates are needed: *orientation* and *in_circle*



ROBUSTNESS IN CGAL

Numerical robustness

Recall: Robustness is the ability of a computer system to cope with errors during execution

Exact Geometric Computing $\neq$ exact arithmetics

- Idea: produce “correct” results, in spite of intermediate roundoff errors.
- Example: *If three lines meet in one point, they will do so in CGAL as well, and if a fourth line misses this point by 1.0×10^{-380} , then it also misses it in CGAL*

Source: <http://www.cgal.org/exact.html>

ROBUSTNESS IN CGAL

Numerical robustness

Recall: Robustness is the ability of a computer system to cope with errors during execution

Exact Geometric Computing $\neq$ exact arithmetics

- Idea: produce “correct” results, in spite of intermediate roundoff errors.
- Example: *If three lines meet in one point, they will do so in CGAL as well, and if a fourth line misses this point by 1.0×10^{-380} , then it also misses it in CGAL*

CGAL implements robustness using **arithmetic filters**

Source: <http://www.cgal.org/exact.html>

ARITHMETIC FILTERS

To determine $\text{sign}(\mathbf{P}(\mathbf{x}))$

Consider

- 1) $P^a(x)$: approximate (say, floating point) evaluation of $P(x)$
- 2) Error estimation ε

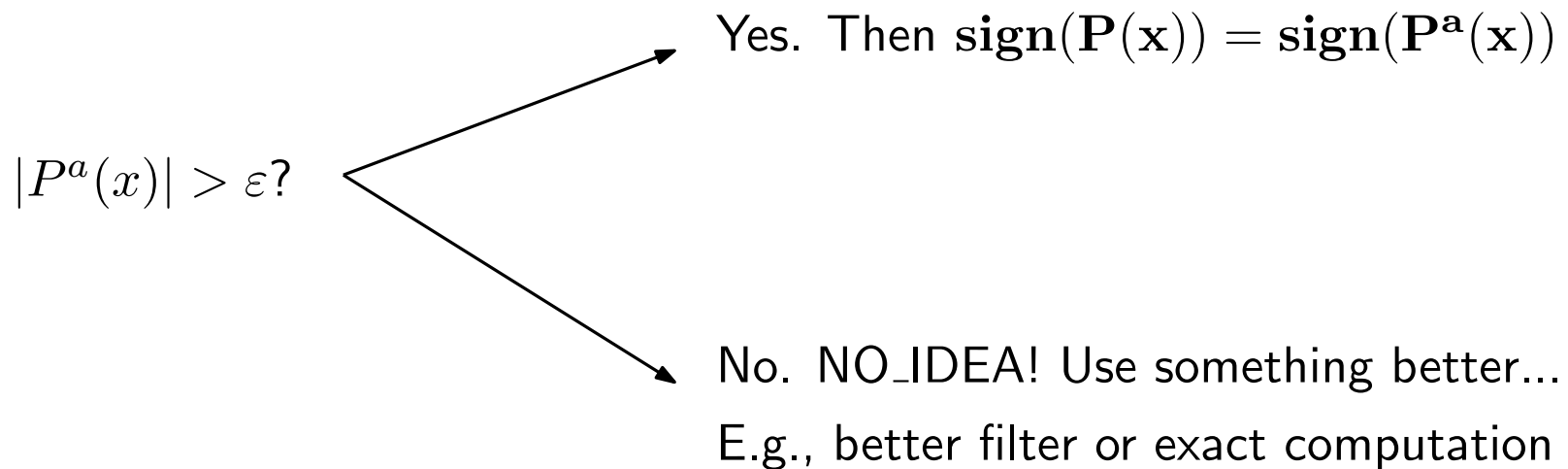
ROBUSTNESS AND CGAL

ARITHMETIC FILTERS

To determine $\text{sign}(\mathbf{P}(\mathbf{x}))$

Consider

- 1) $P^a(x)$: approximate (say, floating point) evaluation of $P(x)$
- 2) Error estimation ε



ROBUSTNESS AND CGAL

ARITHMETIC FILTERS

To determine $\text{sign}(\mathbf{P}(\mathbf{x}))$

Consider

1) $P^a(x)$: approximate (say, floating point) evaluation of $P(x)$

2) Error estimation ε

Depending on how ε is estimated, filter can be *static*, *semi-static* or *dynamic*

$|P^a(x)| > \varepsilon?$

Yes. Then $\text{sign}(\mathbf{P}(\mathbf{x})) = \text{sign}(\mathbf{P}^a(\mathbf{x}))$

No. NO_IDEA! Use something better...
E.g., better filter or exact computation

ROBUSTNESS AND CGAL

EXAMPLE OF STATIC FILTER

Approximate non guaranteed version

```
int orientation(double px, double py,
               double qx, double qy,
               double rx, double ry)
{
    double pqx = qx - px,  pqy = qy - py;
    double prx = rx - px,  pry = ry - py;

    double det = pqx * pry - pqy * prx;

    if (det > 0)  return 1;
    if (det < 0)  return -1;
    return 0;
}
```

Source: slides by M. Teillaud

ROBUSTNESS AND CGAL

EXAMPLE OF STATIC FILTER

Code with static filtering (for entries **bounded by 1**):

```
int filtered_orientation(double px, double py,
                        double qx, double qy,
                        double rx, double ry)
{
    double pqx = qx - px,  pqy = qy - py;
    double prx = rx - px,  pry = ry - py;

    double det = pqx * pry - pqy * prx;

    const double E = 1.33292e-15;

    if (det > E)  return 1;
    if (det < -E) return -1;

    ... // can't decide => call the exact version
}
```

Source: slides by M. Teillaud

DYNAMIC FILTERS IN CGAL

Based on **interval arithmetic**

- Floating point operations replaced by operations on intervals
- x replaced by $[\underline{x}; \overline{x}]$
 - At each operation, the interval contains the exact value

DYNAMIC FILTERS IN CGAL

Based on **interval arithmetic**

- Floating point operations replaced by operations on intervals
- x replaced by $[\underline{x}; \overline{x}]$
 - At each operation, the interval contains the exact value

Operations on intervals based on rounding modes of IEEE 754 standard

For example:

- $X + Y \rightarrow [\underline{x} + \underline{y}; \overline{x} + \overline{y}]$
- $X - Y \rightarrow [\underline{x} + \overline{y}; \overline{x} - \underline{y}]$
- $X \times Y \rightarrow [\min(\underline{x} \times \underline{y}, \underline{x} \times \overline{y}, \overline{x} \times \underline{y}, \overline{x} \times \overline{y}); \max(\underline{x} \times \underline{y}, \underline{x} \times \overline{y}, \overline{x} \times \underline{y}, \overline{x} \times \overline{y})]$

ROBUSTNESS AND CGAL

DYNAMIC FILTERS IN CGAL

Based on **interval arithmetic**

- Floating point operations replaced by operations on intervals
- x replaced by $[\underline{x}; \overline{x}]$
 - At each operation, the interval contains the exact value

Operations on intervals based on rounding modes of IEEE 754 standard

For example:

- $X + Y \rightarrow [\underline{x} + \underline{y}; \overline{x} + \overline{y}]$
- $X - Y \rightarrow [\underline{x} + \overline{y}; \overline{x} - \underline{y}]$
- $X \times Y \rightarrow [\min(\underline{x} \times \underline{y}, \underline{x} \times \overline{y}, \overline{x} \times \underline{y}, \overline{x} \times \overline{y}); \max(\underline{x} \times \underline{y}, \underline{x} \times \overline{y}, \overline{x} \times \underline{y}, \overline{x} \times \overline{y})]$

Inclusion property If $[\underline{x}; \overline{x}] \cap [\underline{y}; \overline{y}] = \emptyset$ then we can decide whether $X < Y$ or $X > Y$.
Otherwise, we cannot decide (the filter fails)

ROBUSTNESS AND CGAL

EFFICIENCY

Running time examples based on CGAL 3.1 (old!)

3D Delaunay triangulation

CGAL-3.1-l-124

Pentium-M 1.7 GHz, 1GB

g++ 3.3.2, -O2 -DNDEBUG

1.000.000 random points

Simple_Cartesian< double >

48.1 sec

Simple_Cartesian< MP_Float >

2980.2 sec

Filtered_kernel (dynamic filtering)

232.1 sec

Filtered_kernel (static + dynamic filtering)

58.4 sec

Source: slides by M. Teillaud

ROBUSTNESS AND CGAL

EFFICIENCY

Running time examples based on CGAL 3.1 (old!)

3D Delaunay triangulation

CGAL-3.1-l-124

Pentium-M 1.7 GHz, 1GB
g++ 3.3.2, -O2 -DNDEBUG

1.000.000 random points

Simple_Cartesian< double > 48.1 sec

Simple_Cartesian< MP_Float > 2980.2 sec

Filtered_kernel (dynamic filtering) 232.1 sec

Filtered_kernel (static + dynamic filtering) 58.4 sec

Source: slides by M. Teillaud

Using filtered arithmetics in CGAL is easy with predefined kernels

```
#include <iostream>
#include <CGAL/Exact_predicates_inexact_constructions_kernel.h>
#include <CGAL/convex_hull_2.h>

typedef CGAL::Exact_predicates_inexact_constructions_kernel K;
typedef K::Point_2 Point_2;

int main()
{
    Point_2 points[5] = { Point_2(0,0), Point_2(10,0), Point_2(10,10), Point_2(6,5), Point_2(4,1) };
    Point_2 result[5];
```

REFERENCES

- L. Kettner, K. Mehlhorn, S. Pion, S. Schirra, C. Yap: *Classroom Examples of Robustness Problems in Geometric Computations*, Computational Geometry: Theory and Applications 40:61–78, 2008.
- H. Bronnimann, C. Burnikel, S. Pion *Interval Arithmetic Yields Efficient Dynamic Filters for Computational Geometry*, Discrete Applied Mathematics 109:25–47, 2001.
- www.cgal.org